Лабораторная работа №5

Функциональные возможности Python: функции высшего порядка, итераторы и генераторы

1. Цель работы

Применить lambda, map, filter, sorted, генераторы и функции высшего порядка при обработке коллекции объектов учебных курсов.

2. Постановка индивидуального задания

Вариант 12 — «Учебные курсы». Реализовать обработку курсов средствами функционального программирования.

Программа должна:

– Обрабатывать курсы с преподавателем, часами, ценой и количеством студентов.

– Выполнить фильтр по цене, получение названий, сортировку по числу студентов и среднюю цену.

– Проверить переполненные курсы и создать генератор отбора по преподавателю.

3. Краткие теоретические сведения

Функция высшего порядка принимает функцию как аргумент или возвращает функцию. filter() использует предикат отбора, map() преобразует элементы, sorted(..., key=...) задаёт порядок сортировки.

Анонимные функции lambda используются, когда логика краткая и нужна только в месте вызова. List comprehension создаёт список вычисленных значений, а any() определяет, есть ли хотя бы один объект, удовлетворяющий условию.

Генератор с оператором yield выдаёт элементы по запросу и не материализует весь результат сразу. В проекте teacher_courses() последовательно возвращает только курсы конкретного преподавателя. По сравнению с готовым списком генератор вычисляет элементы лениво.

4. Исходный код программы

Файл models.py

```
class Course:
    def __init__(self, name, teacher, hours, price, students):
        if hours <= 0 or price < 0 or students < 0:
            raise ValueError("Неверные данные курса")
        self.name = name
        self.teacher = teacher
        self.hours = hours
        self.price = price
        self.students = students

    def __repr__(self):
        return f"{self.name}: {self.price} руб., {self.students} студентов"


class OnlineCourse(Course):
    def format(self):
        return "Онлайн"


class ClassroomCourse(Course):
    def format(self):
        return "В аудитории"
```

Файл operations.py

```
def filter_objects(objects, predicate):
    return list(filter(predicate, objects))


def transform_objects(objects, operation):
    return list(map(operation, objects))


def sort_objects(objects, key_function):
    return sorted(objects, key=key_function)


def average_price(courses):
    if not courses:
        raise ValueError("Нет курсов")
    return sum(c.price for c in courses) / len(courses)
```

Файл generators.py

```
def teacher_courses(courses, teacher):
    for course in courses:
        if course.teacher == teacher:
            yield course
```

Файл main.py

```
from models import OnlineCourse, ClassroomCourse
from operations import filter_objects, transform_objects, sort_objects, average_price
```

```
from generators import teacher_courses


def main():
    courses = [OnlineCourse("Python", "Иванов", 40, 5000, 18),
               ClassroomCourse("Математика", "Петров", 60, 6000, 35),
               OnlineCourse("SQL", "Иванов", 30, 4000, 12)]
    try:
        price = float(input("Максимальная стоимость: "))
        capacity = int(input("Максимум студентов на курс: "))
        if price < 0 or capacity <= 0:
            raise ValueError("Стоимость неотрицательна, вместимость положительна")
        print("По стоимости:", filter_objects(courses, lambda c: c.price <= price))
        print("Названия:", transform_objects(courses, lambda c: c.name))
        print("По числу студентов:", sort_objects(courses, lambda c: c.students))
        print("Средняя стоимость:", average_price(courses))
        print("Есть переполненные:", any(c.students > capacity for c in courses))
        print("Форматы:", [c.format() for c in courses])
        teacher = input("Преподаватель: ")
        for course in teacher_courses(courses, teacher):
            print(course)
    except ValueError as error:
        print("Ошибка:", error)


if __name__ == "__main__":
    main()
```

5. Примеры работы программы в виде снимков экрана

На рисунке 5.1 представлены отбор, сортировка, вычисление среднего значения и результат генератора по преподавателю.

```
python main.py
Максимальная стоимость: 5500
Максимум студентов на курс: 25
По стоимости: [Python: 5000 руб., 18 студентов, SQL: 4000 руб., 12 студентов]
SQL: 4000 руб., 12 студентов]
Названия: ['Python', 'Математика', 'SQL']
По числу студентов: [SQL: 4000 руб., 12 студентов, Python: 5000 руб., 18
студентов, Математика: 6000 руб., 35 студентов]
SQL: 4000 руб., 12 студентов, Python: 5000 руб., 18 студентов, Математика:
6000 руб., 35 студентов]
Средняя стоимость: 5000.0
Есть переполненные: True
Форматы: ['Онлайн', 'В аудитории', 'Онлайн']
SQL: 4000 руб., 12 студентов
```

Рисунок 5.1 – Обработка данных учебных курсов

6. Вывод

Освоены функции высшего порядка, лямбда-выражения, сортировка, включения списков и генератор с yield на примере учебных курсов. Контрольный запуск подтвердил правильность фильтрации и агрегирования данных.

Выполнил	Пахомов Максим	21.10.2026
Принял	Елкин Н. С.	